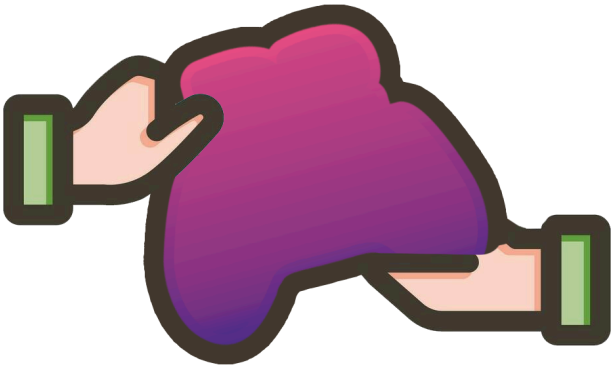
Dev2dev
UNITY TEMPLATE DOCUMENTATION



SCENES

MainMenuScene

Main Menu has functional buttons for Start Game, Settings, Credits and Quit Game. Settings is a prefab so you can use it in other scenes as well. It automatically loads the audio settings once enabled. Main Menu calls LoadSettings() on Start() so the audio volumes are adjusted.

LoadingScene

This scene is added to any scene you are playing in play mode additively after the scene loads thanks to this snippet inside [LoadingManager.cs]

```
[RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.AfterSceneLoad)]
static void Init()
{
    string persistentSceneName = "LoadingScene";
    if (!SceneManager.GetSceneByName(persistentSceneName).isLoaded)
    {
        SceneManager.LoadScene(persistentSceneName, LoadSceneMode.Additive);
    }
}
```

This scene handles fading in and out when switching between scenes.

GameScene

Where the actual game is happening.
It has several Manager classes and a pause button that would open the settings popup and pause the game(added this so in WebGL people can pause the game without hitting Esc).

MANAGERS

GameManager

The ultimate singleton class. It's intended to add any other manager references under this object to have global access.
It also has a way of controlling the game phases if you want to add some logic when switching between phases like GAMEPLAY to CUTSCENE.

UIManager

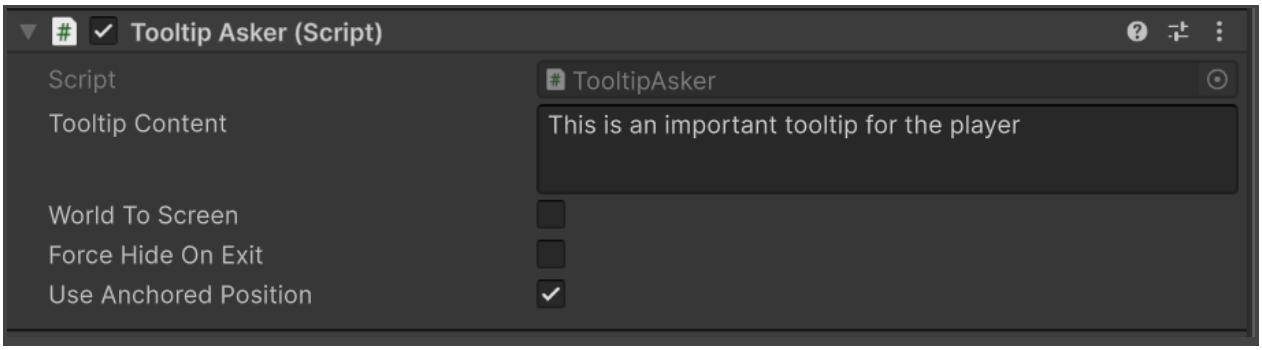
Nothing major here. It is the main Canvas. It helps the TooltipManager to understand the width and height of the window. It's also intended to hold the references of UI elements and handle the UI logic.

LoadingManager

To load a scene using fade in and fade out, you can simply call
```
LoadingManager.LoadScene("sceneName");
```
To fade out manually without loading a scene:
```
LoadingManager.ForceFadeOut();
```
To fade in manually:
```
LoadingManager.ForceFadeIn();
```

TooltipManager

The only thing you need to do for tooltips is to add a TooltipAsker component to the object you want to show tooltip for.



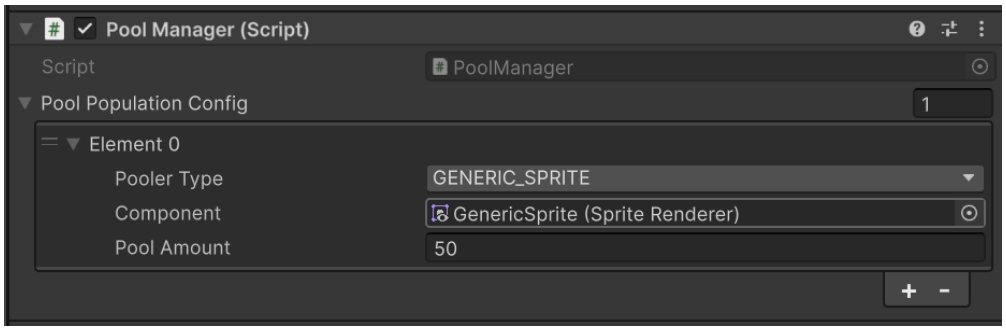
Use "World to Screen" for world objects and "Use Anchored Position" for UI objects.

PoolManager

You should add enums of all the objects you want to pool:

```
10 usages   3 exposing APIs
public enum PoolerType
{
    NONE,
    GENERIC_SPRITE
}
```

And then add them to the list in the PoolManager object, so they will get populated once the scene loads:



Finally, you can do this instead of instantiating:

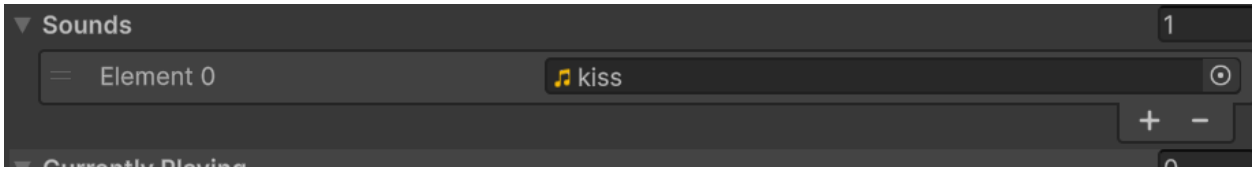
```
SpriteRenderer spriteFromPool = PoolManager.DequeueObejct<SpriteRenderer>( key: PoolerType.GENERIC_SPRITE);
spriteFromPool.transform.position = Random.insideUnitCircle;
```

And this instead of destroying:

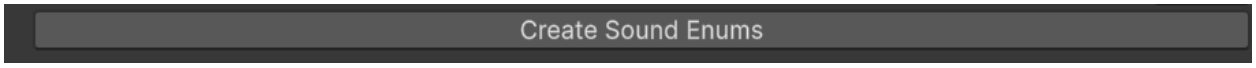
```
PoolManager.EnqueueObject(spriteFromPool,PoolerType.GENERIC_SPRITE);
```

SoundManager

Add any sound you want to play under the Sounds array in the SoundManager object.



And then click this button:



It will automatically create enums for all the sounds in the array:

```
10 usages   1 exposing API
public enum SoundType
{
 KISS,
}
```

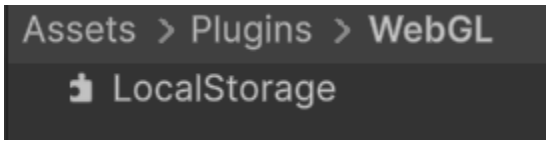
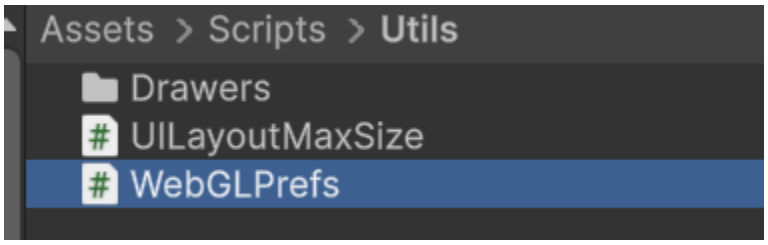
And then you can just go ahead and do this anywhere you like:

```
(SoundType sound, float volume = 1F, float pitch = 1F, string mixerName = "SFX") : void
SoundManager.Instance.PlayOneShot(SoundType.KISS, volume: 1f, pitch: Random.Range(0.8f,1.2f));
```

UTILS

WebGLPrefs

On itch, when you update the game it creates a new instance of it and PlayerPrefs entries get reset. WebGLPrefs uses another approach where it creates a local storage and always saves and loads it there.

So instead of PlayerPrefs, you can use WebGLPrefs for persistent save files on itch.

This is handled by 2 files: